Towards Running OpenWhisk Actions in Firecracker MicroVMs Using Weave Ignite

Erika Hunhoff, Rhett Hanscom, and Justin Reiss
CSCI5573 Advanced Operating Systems, Fall 2020
University of Colorado Boulder

1 Introduction

In serverless computing, serverless providers take on many responsibilities of datacenter management (scaling, scheduling, isolation, etc.) allowing serverless application developers to focus primarily on their applications. Although serverless computing is a fairly new model of cloud computing, it is already a billion-dollar industry [10].

A serverless *function* is typically run in response to a *trigger*, such as a HTTP request or timer. The function runs in an *execution environment*, which isolates it from other functions running on the same host. The serverless framework must quickly schedule the serverless function, instantiate the execution environment, and initialize the execution environment to run that particular function before the function can be executed. Most serverless functions are quite short-lived: 50% of functions have an average duration of less than 1 second [16]. If the serverless infrastructure is not carefully designed, latency incurred by the serverless framework in managing execution environments can easily dominate serverless function execution time.

Thus, choosing an appropriate type of execution environment and defining how to manage execution environments are vitally important pieces of serverless architecture design. Execution environments must balance the need for isolation (safety from side channels attacks, tenant isolation, etc.), low latency operations (start, stop, etc.), and performance (memory, CPU, and other overheads). In research and industry, there exist numerous approaches to creating serverless execution environments. However, the most popular open source serverless frameworks use containers [12, 13]. It would be useful for the serverless research community to have access to open source serverless frameworks that use virtual machines. To address this need, we work towards integrating Firecracker microVMs into OpenWhisk using Weave Ignite [13–15, 17]. Firecracker is an open source project created by Amazon, and it is used in production for cloud offerings such as AWS Lambda and AWS Fargate [1]. Weave Ignite is a tool used to orchestrate Firecracker microVMs in a fashion similar to Docker [17].

First, we discuss related work in section 2. OpenWhisk is a complex project, so we limit the scope of this project to providing Firecracker support for the JavaScript OpenWhisk runtime only, on the standalone deployment of OpenWhisk. Details of our design are provided in Section 3. Section 4 discusses experimental methodology, and Section 4.1 describes results from preliminary benchmarks. We conclude in Section 5 with discussion and future work.

2 Related Work

2.1 Serverless Execution Environments

There are three common types of execution environments used in production serverless systems: language-based, container-based, and virtual machine-based. A *language-based* execution environment uses virtualization within a language virtual machine (such as the Java Virtual Machine) to isolate a function from the host and from other serverless functions. A production example of language-based execution environments is CloudFlare Workers [4]. Language-based execution environments tend to be light-weight and responsive. However, this method only allows serverless functions to be written in particular languages (*e.g.,* CloudFlare Workers supports only functions written in JavaScript).

To support serverless functions comprised of arbitrary binaries, *container-based* or *virtual machine-based* execution environments are needed. IBM Cloud Functions uses Docker containers as an execution environment [9]. OpenWhisk - an open-source serverless framework initially released by IBM - also uses Docker containers [13]. cContainer-based execution environments tend to be relatively light-weight and allow the execution of arbitrary binaries. Another benefit is that there exist mature container and container orchestration software (*e.g.,* Docker, Kubernetes). Generally, containers use operating system mechanisms such as *cgroups*, *namespaces*, *chroot*, and *seccomp-bpf* to isolate processes in the container from activity outside of the container, and to limit the resources

the container can access. In a serverless setting, the main drawback of containers is that each container shares the host operating system kernel, which presents a large attack surface. While access to kernel functionality can be controlled by utilities such as *seccomp-bpf*, limiting system calls can render the execution environment unusable for unmodified binaries.

Amazon AWS Lambda and Mircosoft Azure Serverless both use *virtual machine-based* execution environments, citing client privacy and security as the driving factor for this choice [1, 2]. Historically, the cost (resource usage, latency, etc.) of virtualization was too high to be considered for fast, ephemeral workloads such as serverless. However, improvements in virtualization technologies make this a competitive approach.

While other approaches to serverless executions environments exist, such as libraryOS [3] and microkernels [6] approaches, they have not seen wide adoption at this time. These approaches also may not support running unmodified binaries.

2.2 Firecracker

In 2018 Amazon open sourced their Firecracker microVM project. Firecracker is used for both AWS Lambda and AWS Fargate, and so has been tested in production at scale. Unlike other virtual machine offerings, Firecracker was built specifically with serverless workloads in mind [1]. Here, we give a high-level description of Firecracker. As the orchestration tool that Amazon uses to manage Firecracker microVMs remains proprietary, the next section also discusses several open source projects that were created to orchestrate Firecracker mircoVMs.

Firecracker is a virtual machine monitor (VMM) used to create microVMs to service ephemeral workloads such as serverless functions. Firecracker is built upon KVM. Initial code for Firecracker began with *crosvm* (Google's Chrome OS Virtual Machine Monitor), but Firecracker's designers removed and added capabilities based on the particular goals of Firecracker and so Firecracker differs substantially from crosvm. There is a one-to-one mapping between a Firecracker process and a microVM, and each microVM runs in a jailed environment (using chroot, seccomp-bpf, and namespaces) as a fail-safe against a process gaining information about the host outside of the microVM. Firecracker accepts commands via an HTTP/REST interface. In benchmarks, it was shown that Firecracker isolated multiple microVMs from a performance standpoint, demonstrated very fast boot times, and has a low memory overhead per microVM. However, Firecracker did not perform as well as competing virtualization options on I/O. For a detailed description of Firecracker and Firecracker's capabilities, we defer to [1].

Weave Ignite is a platform which manages Firecracker microVMs and is built to present an interface similar to the Docker command line interface. Additionally, Weave Ignite has the ability to create and run a Firecracker microVM based on a compatible Docker/OCI image, and provides a way to manage those images. An OCI image is considered compatible if the image contain an init process. To support creation of these images, Weave Ignite provides a variety of compatible images (Ubuntu, CentOS, etc.) and kernels (the default is Linux 4.19) that can be used as a base for custom images. To customize an image, a user can create a Dockerfile and use docker to build the image. Thus, Weave Ignite is a good fit for applications which were formerly Docker-based looking to move towards Firecracker microVMs. Weave Ignite is still a relatively new project (considered to be in the alpha phase of development) [17].

FireKube is a utility that provides a GitOps cluster management interface atop of Weave Ignite. Consequently, it could potentially be used to extend the functionality of Weave Ignite towards applications deployed over multiple nodes. [8]. We also note that although Kata Containers does support the creation of FireKube microVMs, support for Firecracker in Kata Containers is still limited [11].

firecracker-containerd uses containerd to create Firecracker microVMs. We hesitate to use this project in our implementation because it is quite new and only provides limited functionality (*e.g.,* it is limited to only creating several containers at once). It is also worth noting that the key to firecracker-containerd's design is to run every Firecracker microVM within a Docker container. From our point of view, this seems unnecessary since Firecracker microVMs are already run within a jailed environment [7].

3 Design

This project revolves around three technologies: OpenWhisk, Firecracker, and Weave Ignite. We chose OpenWhisk for our serverless framework because it is mature, open source, and often used by researchers. We chose Firecracker for our execution environment because it has been tested at scale and was tailored specifically for ephemeral workloads. We chose Weave Ignite as a wrapper for Firecracker because the Weave Ignite command line interface provides mechanisms to invoke and manage Firecracker microVMs that are similar to what is provided by the Docker command line interface. Additionally, Weave Ignite provides a mechanism to create a Firecracker microVM from a Docker image - aiding in the transition from Docker OpenWhisk runtime images to Firecracker OpenWhisk runtime images.

Additionally, there is a proof-of-concept published to GitHub by Chetan Mehrotra for deploying microVMs to run JavaScript functions with OpenWhisk. Although the implementation does illustrate some of the important factors when considering the integration of OpenWhisk with these tools, it is still preliminary and comment history indicates that others have struggled to run the proof-of-concept suc-

cessfully [15]. In section 3.1 we will describe our overall goals for this project. Since we base our design in part on the the proof-of-concept, Section 3.2 describes the original proof-of-concept. Section 3.3 describes our implementation.

3.1 Design Goals

Our primary goals are to run serverless functions in microVMs using OpenWhisk and to benchmark the performance as compared to unmodified OpenWhisk with Docker. While our eventual goal is to create and run OpenWhisk deployments that use Firecracker at scale, we've limited our current design scope to 1), JavaScript functions; and 2), the standalone version of OpenWhisk. Additionally, we limit our benchmarks to serial invocations of functions.

Working only with the standalone version of OpenWhisk restricts the usefulness of this project because it precludes large scale deployments of OpenWhisk. However, we posit that work on the standalone version is a necessary step towards larger deployments. In Section 5.2 we discuss potential paths towards more realistic deployments.

Although we have limited this work to JavaScript functions, future researchers would need only to create a Firecracker-compatible Docker/OCI image from the original OpenWhisk Docker image in order to enable support for other programming languages. Generally, the only reason why the existing OpenWhisk Docker images are not compatible is that they must have an init process. As the process for creating a compatible Docker image for Weave Ignite is well documented [17], we focus our energies elsewhere.

3.2 Previous Work

The proof-of-concept uses a design whereby each Firecracker microVM was shadowed by a corresponding Docker container [15]. This design was chosen due to limitations of the Ignite CLI (more specifically because the IP addresses of Firecracker microVMs could not be fetched via the Ignite CLI at the time when the proof-of-concept was created). The proof-of-concept was created by modifying the OpenWhisk `invoker` component. The `invoker` is responsible for performing tasks that make it possible to invoke a function in an execution environment. To enable the `invoker` to control Firecracker microVMs via Ignite, the proof-of-concept defined an `IgniteClient`, `IgniteAPI`, `IgniteContainer`, and `IgniteContainerFactory`. These classes provide an interface between the invoker's container pool abstraction and implementation-specific details. The proof-of-concept also created a Firecracker microVM image based on the Docker image used for running JavaScript functions.

3.3 Current Implementation

The proof-of-concept was incredibly useful in informing our design, but as we progressed we diverged from the proof-of-concept in several notable ways. Upon experimenting with the proof-of-concept, we found we could not get the unmodified design to successfully run a function. It also immediately became apparent that a design that relies on a 1-1 shadowing of a Docker container to a Firecracker microVM would cause performance degradation. The proof-of-concept was also branched off of the OpenWhisk repository when it was still in an incubation stage, which posed a problem because we wanted our solution to target the first stable release of OpenWhisk, 1.0.0. Similarly, the OpenWhisk JavaScript runtime used in the Firecracker-compatible Docker image was outdated. Thus, our initial goals were to get the prototype running, remove Docker shadowing, and to port the modified solution to an updated Docker image and OpenWhisk version.

We first debugged the proof of concept. Many of the errors we were seeing seemed to be related to the Docker shadowing. Consequently, removing the shadowing helped to debug and simplify the implementation. Luckily, Weave Ignite had progressed since the time the proof-of-concept was created, so we were able to remove Docker shadowing without incident. However, we did notice some differences in expressivity and function between the Docker CLI and the Ignite CLI which will be covered in more detail in Section 5.1.

The proof-of-concept noted that logging was not working correctly. When we removed the Docker shadowing, this failure to collect logs from the Firecracker microVM introduced behavior that prevented the use of warmed containers. As logs could not be fetched after an invocation of a function in a Firecracker micoVM, the `invoker` assumed that the container had experienced an error and should not be used again. We addressed this issue by using the `ignite log` command to fetch logs. However, this is a stop-gap measure used to ensure we can use warmed containers in our benchmarks. We address logging in more detail in Section 5.1.

We successfully ported our solution to work in OpenWhisk 1.0.0, and updated the image to use the current default JavaScript OpenWhisk Docker image: `openwhisk/action-nodejs-v10:1.16.0`. In summary, we have achieved and lightly tested these first three goals for the simple use case of running a single JavaScript function via a Firecracker microVM.

4 Methodology

All of our benchmarks are aimed towards evaluating the performance of individual containers or microVMs. Hence, each test invocation was initiated serially. While the Firecracker paper compares Firecracker against other virtualization technologies such as QEMU, this work presents a comparison which is fundamentally different by comparing Firecracker to

Docker containers.

We made one modification to the default OpenWhisk+Docker configuration. Firecracker microVMs are only configurable be specifying a number of CPUs. In contrast, the OpenWhisk `invoker` configures docker containers based on CPU shares. We modified the default OpenWhisk+Docker to use the same number of CPUs we allocated to each Firecracker microVM. Ubuntu 18.04 was used as the host operating system for this experiment, and an Ubuntu-based image was used for the operating system within the Firecracker microVMs. It is worth noting that we did not attempt to match the kernel versions of the Firecracker microVMs to the host OS (which is what the Docker containers use).

As stated previously, our implementation is based on OpenWhisk v1.0.0. Both the Ignite/Firewhisk and Docker tests use the same standalone OpenWhisk build file; the container engine and logging mechanism are toggled between the two via a configuration file. We use Docker version 19.03.6 and Ignite version 0.7.1 (backed by Firecracker 0.21.1). Barring the differences in images, we used identical runtime configurations for both Docker and Firecracker/Ignite.

We use CloudLab to provision four nodes consisting of eight-core Intel Xeon D-1548 at 2.0 GHz, 64GB ECC RAM (4x 16 GB DDR4-2133 SO-DIMMs), 256 GB NVMe flash storage disk, and dual-port Mellanox ConnectX-3 10 GB NICs (PCIe v3.0, 8 lanes) [5]. We allocated two CloudLab nodes for each of the Docker and Firewhisk tests, for a total of four nodes. On two of the nodes (one Firecracker and one Docker) we ran each experiment once; on the other two nodes, we ran each experiment twice. Each experimental run consisted of 20 iterations, so this gives us a total of 60 iterations for each of Docker and Firecracker per benchmark. Each bar chart uses results from all 20 iterations (1 node); each CDF graph uses results from all 60 iterations.

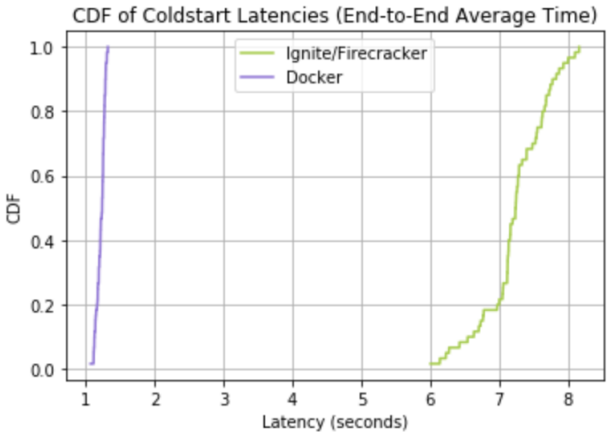


Figure 1: CDF of end-to-end latency of a coldstarts.

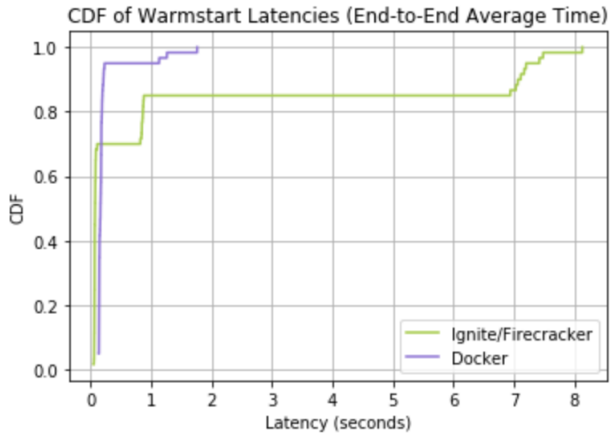


Figure 2: CDF of end-to-end latency of warmstarts.

4.1 Evaluation

We present benchmarks covering coldstart and warmstart (4.2) latencies, CPU performance (4.3), system call overheads (4.4), network performance (4.5), and file operations (4.6). Due to time constraints, we leave benchmarks on memory utilization for future work.

4.2 Coldstart and Warmstart Benchmarks

The coldstart benchmark, illustrated in Figure 1, is used to gauge the time it takes to call a simple OpenWhisk action which immediately returns. The term *coldstart* refers to the overhead of creating a new Docker container or Firecracker microVM, and we force a coldstart on each iteration of this benchmark.

A *warmstart* refers to when a container of microVM is reused for multiple invocations. Figure 2 measures the end-to-end time of a simple function that returns without taking any action in a warmed container or microVM.

In both benchmarks, Docker shows a lower end-to-latency. For coldstarts, Docker show significantly smaller latency in all cases. For warmstarts, Firecracker is competitive with Docker in the general case; however, the 80% tail latency of Firecracker drags on to over eight seconds! As we see such high variance in warmstarts, which the rest of our benchmarks use, we do not show additional end-to-end timing graphs. This is done because variance here tends to overshadow the variance of the operations performed within the body of the OpenWhisk function.

These results are very different than the results shown by Firecracker benchmarks conducted elsewhere. Our benchmark is not very granular; end-to-end times include the performance of our OpenWhisk `invoker` modifications, the performance of the Ignite CLI, and performance of Firecracker for our chosen microVM image. Additional benchmarking

is needed to determine what is causing such high coldstart latency and such a long warmstart tail latency. One major difference between our experiment and other Firecracker experiments is that we created our microVM image without much performance tuning.

4.3 CPU Benchmark

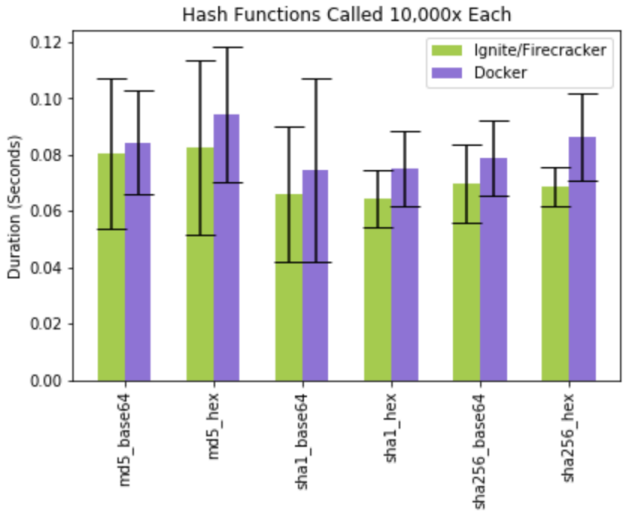


Figure 3: Average time spent to call various hash functions 10,000 times.

Our simple CPU-focused benchmark, pictured in Figure 3, hashes a string based on three different hashing functions 10,000 times, each as both hexadecimal and base64. This results in 60,000 hash operations each time the benchmark is run.

Performance of the individual hashing operations on the CPU were comparable between Firecracker microVMs and Docker containers, with microVMs finishing operations slightly faster than Docker containers in all cases (pictured in Figure 3). While these preliminary results are promising, there is a considerably large amount of variance present in reported times, as illustrated within Figure 3 within the depicted standard deviation.

4.4 System Call Benchmark

The system call benchmark repeatedly initiates function calls to `Date.now()` in JavaScript, which in turn invokes the `clock_gettime` system call. This process was repeated 1,000,000 times for each initiation of the benchmark, and is depicted in Figure 4. Here we find Firecracker to show slightly less overhead when executing this system call over number calls, with each Docker and Firecracker exhibiting

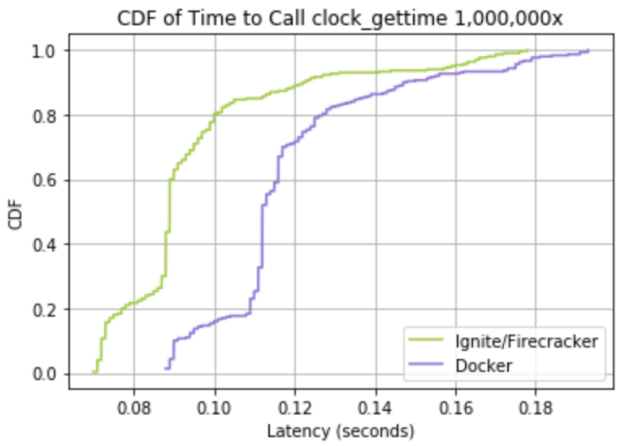


Figure 4: Average time spent to call the *clock_gettime* system call 1,000,000 times.

similar amounts variance within these reported system calls. We show a CDF here instead of a bar chart to illustrate that the CDFs of Docker and Firecracker are similarly shaped, but with Docker's shifted to the right.

4.5 Network Benchmark

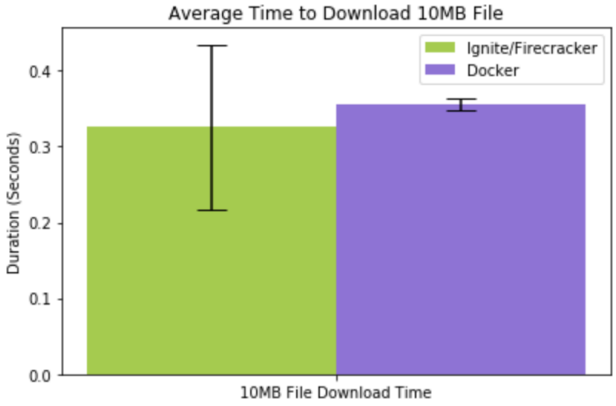


Figure 5: Average time spent to downloading a 10MB file across many occurrences on both Firecracker microVM and Docker OpenWhisk deployments.

The network benchmark, shown in Figure 5, measured the time it took to download a 10MB text file five times. This benchmark, along with all others, was run numerous times to ensure sufficient data as to minimize anomalies. While overall download speeds were comparable, and even slightly faster within the Firecracker microVM on average, the standard deviation of speed was far greater for the Firecracker microVMs

when compared to the more stable download speeds seen in the Docker image download tests. In fact, while worst-case download speeds for Docker never reached 0.6 seconds, in a small number of cases Firecracker microVMs took almost 1.4 seconds to finish the 10MB download.

Future work would expand the network tests to include upload speed tests and possibly latency as well. It would be useful to use standard tools such as iperf3 for future tests. For completeness, in the future we would like to include similar metrics from operations run directly on the serverless host.

4.6 File Creation and I/O Benchmarks

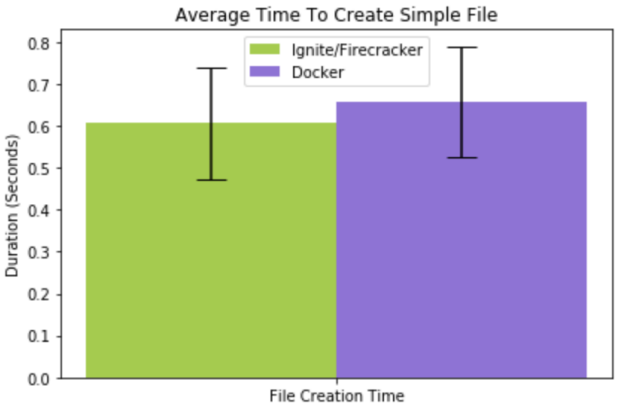


Figure 6: Average time spent creating a file. A CDF representing this same benchmark is found in Figure 7.

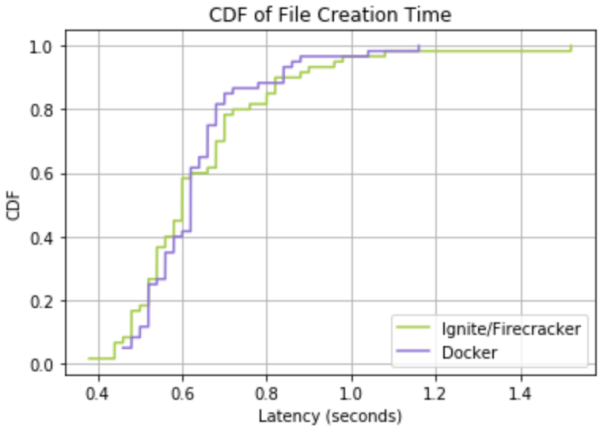


Figure 7: Average time spent creating a file. A bar graph representing this same benchmark is found in Figure 6

We designed two simple benchmarks measure file I/O operations. The first is a file creation benchmark (Figure 6) which

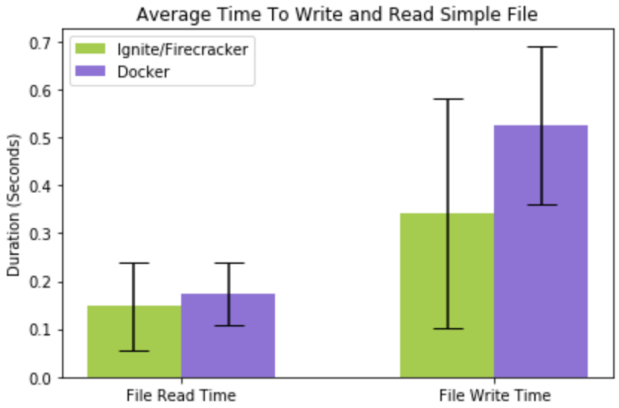


Figure 8: Average time spent creating, writing to, and reading back a text file.

creates a text file and writes a small string (24 bytes) to the file, before moving on to the next file creation. Each function does this 500 times. These results are expanded with greater complexity in the file read/write benchmark, in Figure 8, which creates a text file, writes a slightly larger string to the file (600 bytes), and then reads the content of the file. The read/write benchmark performs 100 iterations of this per function. All listed values are reported as means across all iterations of the corresponding benchmarks.

We see that Firecracker performs faster than Docker in all three categories (file creation, file read, file write). However, as shown in Figure 7, Firecracker has a substantially longer tail latency than Docker by more than 0.3 seconds.

In future benchmarks, we would like to vary the size of the read/write more substantially, as well as pursue use of most standard tools such as fio. Use of these tools was out of scope due to the limited runtime support provided by the project's current design. Consequently, further work may enable more comprehensive benchmarking in the future.

5 Discussion and Future Work

While our solution works well in the simple use-case we've used in our benchmarks, there are several tasks left before full support in standalone OpenWhisk. In Section 5.1 we cover three major topics related to our current implementation: logging, concurrent invocations and orchestration commands, and pre-warmed containers and pre-fetching images. Then, in Section 5.2, we discuss what is needed to move beyond the standalone deployment of OpenWhisk.

5.1 Future Implementation Tasks

Logging The Docker log engine is robust, and automatically captures the standard output of the process running in

the Docker container and adds it to the container log. The `ignite log` command at first seems similar to the `docker log` command; however, the Ignite log is only the boot log of the Firecracker microVM. The Docker log engine can automatically format logs in a variety of ways, such as json. The Ignite log facility does not have this feature. Docker log timestamps correspond to the clock on the host, making it easier to correlate log messages with OpenWhisk events logged by the host, the Ignite log instead uses timestamps corresponding to the time since boot of the microVM.

While our current `invoker` implementation fetches the boot log via the `ignite log` command, in future work we'd like to create a mechanism to capture standard output of the serverless runtime process inside the microVM (and likely standard error as well). There are several potential methods for implementing this. The first method would be to find a way to pipe output from the process to the virtual machine log directly, and then use `ignite log` unmodified. However, it's unclear how to implement this method. A second option would be to use `ignite cp` command to copy a log file within the microVM to the host file system. Making sure this is safe (*e.g.,* that the serverless function can not replace the log with malware of some kind) and performant are concerns with this approach.

Concurrent invocations and orchestration commands In experimenting with the Ignite CLI, we encountered problems when running concurrently issued commands - especially two commands to import the same image (*e.g.,* `ignite image import`). Also, if the same image is fetched twice, two versions of the image are stored by ignite (instead of indicating it has already been fetched). In the worst case, we found that such commands can seemingly "poison" the Ignite image store with "ambiguous" copies of the same image. These ambiguous copies mean that all future pulls of that image fail, and the image can only be used to create a microVM if it uses an Ignite imageId (which is generated dynamically on import) rather than the image name. We also suspect we had some trouble with concurrently running other Ignite CLI commands, such as `ignite run` (to start a microVM). However, we did not have time to fully debug these issues.

The OpenWhisk `invoker` is designed to run as many tasks as it can concurrently by using Scala Futures to create asynchronous wrappers around commands executed by Docker (or Ignite). At a minimum, to fix the `ignite image import` issue, we would need to add synchronization locks in each location where such a command is run. Future work on this aspect of the project involves fully determining which Ignite commands cannot be run concurrently, and then wrapping those commands with a synchronization mechanism.

Pre-warmed containers and pre-fetching Images The standalone version of OpenWhisk performs pre-flight tests to ensure the system has all necessary pieces installed. It can also be configured to pre-fetch certain images specified in a runtime manifest provided when standalone OpenWhisk is started. The runtime manifest, a json file which specifies information about all of the runtimes supported by OpenWhisk, can also specify what is referred to as `stemCells`. This configuration options specifies if and how OpenWhisk should pre-warm containers before they are needed (to reduce coldstarts).

While we have a preliminary implementation of all of these pieces for Ignite, this implementation does not work due to the concurrency issues mentioned above. Image pre-fetching occurs before the `IgniteClient` (or the `DockerClient`) is instantiated. This makes it harder to correctly synchronize `ignite image import` commands, since that synchronization state is encapsulated in the `IgniteClient` instance used by the `invoker`. Pre-warming containers also appears to directly run commands instead of going through the `DockerClient` (or `IgniteClient`). Successful implementation of both image pre-fetching and pre-warming containers will depend either on changes to Ignite or to Openwhisk. Ignite would need to be modified to be more robust to concurrency. Otherwise, architectural changes to OpenWhisk would be necessary to allow for concurrency mechanisms controlling Ignite image actions to be percolated through to these operations.

Performance and Tuning While the tests in the Firecracker NSDI paper show very promising performance results, it's worth noting that the kernel, image, and method of invocation used in those benchmarks were tuned and optimized carefully. As of now, our implementation uses a default Weave Ignite kernel and operating system image as its basis. Also, we invoke our serverless function via systemd as the init process. There are opportunities to tune the image to the specific use-case of running OpenWhisk functions that will likely improve performance beyond what we currently see in our benchmarks.

Other Tasks Our current implementation was ported to the current version of OpenWhisk but remnants of the structure of older container clients remain in the `IgniteClient` (and related) code. For simplicity, we have also mirrored the `DockerClient` in our code when possible. Future work includes cleaning up our implementation as well as critically evaluating the structure to optimize and fit it to the structure of Ignite.

5.2 Towards Production Deployments

In the Kubernetes-based OpenWhisk deployment (the most popular 'production' OpenWhisk deployment), there are two classes of compute that run in Docker containers: the OpenWhisk infrastructure (couchDB, Kafka, Zookeeper, etc.), and the serverless functions. In the Kubernetes deployment of OpenWhisk, functions are run within Docker containers within Kubernetes pods. We did not attempt to use Weave

Ignite in concert with Kubernetes. One option would be to remove Kubernetes and use a different orchestration tool, such as Firekube (which includes support for both Docker as well as Firecracker through Weave Ignite). Our initial investigation of Firekube shows that Firekube itself is a fairly new project, indicating that creating a Firekube OpenWhisk deployment would be a nontrivial task. We leave these approaches to future work.

6 Conclusion

In this project, we take steps towards creating and benchmarking a standalone version of OpenWhisk that uses Weave Ignite to orchestrate Firecracker microVMs to run the OpenWhisk serverless functions. We use a proof of concept as the base for the project, and we modify that implementation into the current prototype that includes additional reliability and function. We then perform benchmarks comparing our OpenWhisk+Firecracker implementation to unmodified OpenWhisk+Docker.

In the benchmarks we seek to measure performance in areas covering warmstarts, coldstarts, CPU, system calls, networking, and file operations. While Ignite/Firecracker performs well on many operations internal to the function, in most tests we see extended variance (tail latency) in Ignite/Firecracker as compared to Docker. This is particularly notable during warmstarts. While these benchmarks provide an initial glimpse into the performance of Ignite/Firecracker, there is much room for improvement in both the types of benchmarks we can run, as well as our experimental methodology.

Although our implementation was sufficient for the simple benchmarks we ran, it is still incomplete. We look forward to improving our benchmarks and our implementation in the future.

References

[1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434, Santa Clara, CA, February 2020. USENIX Association.

[2] Ricardo Bianchini. Serverless in seattle: Toward making serverless the future of the cloud, Oct 2020.

[3] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. Seuss: Skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems*, EuroSys '20, New York, NY, USA, 2020. Association for Computing Machinery.

[4] Cloudflare workers. https://workers.cloudflare.com/.

[5] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 1–14, July 2019.

[6] Henrique Fingler, Amogh Akshintala, and Christopher J. Rossbach. Usetl: Unikernels for serverless extract transform and load why should you settle for less? In *Proceedings of the 10th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys '19, page 23–30, New York, NY, USA, 2019. Association for Computing Machinery.

[7] firecracker-containerd. https://github.com/firecracker-microvm/firecracker-containerd.

[8] Firekube - fast and secure kubernetes clusters using weave ignite. https://www.weave.works/blog/firekube-fast-and-secure-kubernetes-clusters\-using-weave-ignite.

[9] Ibm cloud functions. https://www.ibm.com/cloud/functions/.

[10] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, Joseph E. Gonzalez, Raluca Ada Popa, Ion Stoica, and David A. Patterson. Cloud programming simplified: A berkeley view on serverless computing, 2019.

[11] Kata cotnainers with firecracker hypervisor. https://github.com/kata-containers/documentation/wiki/Initial-release-of-Kata-Containers-with-\Firecracker-support.

[12] Openfaas. https://www.openfaas.com/.

[13] Apache openwhisk. https://github.com/apache/openwhisk.

[14] Openwhisk issue 4301: add firecracker container factory implementation. https://github.com/apache/openwhisk/issues/4301.

[15] Execute openwhisk action via ignite and firecracker vm 4556. https://github.com/apache/openwhisk/pull/4556.

[16] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218. USENIX Association, July 2020.

[17] Fire up your vms with weave ignite. https://www.weave.works/blog/fire-up-your-vms-with-weave-ignite.